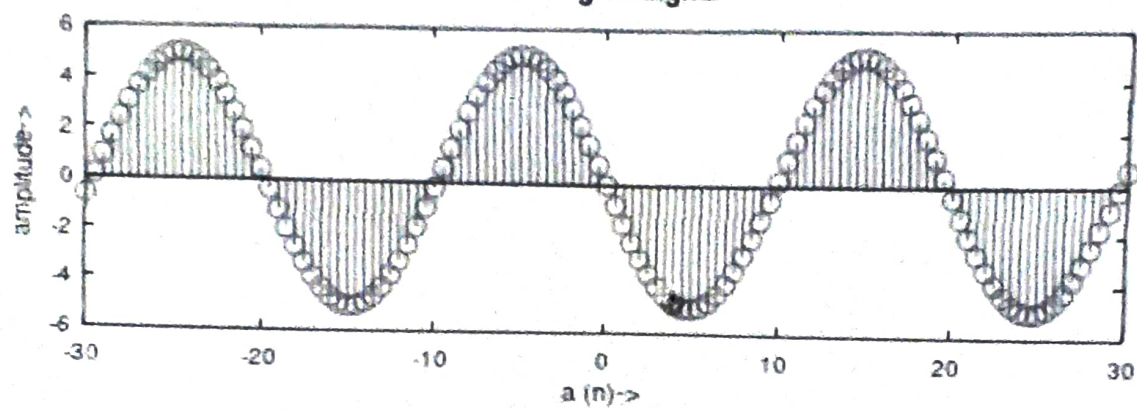


INDEX

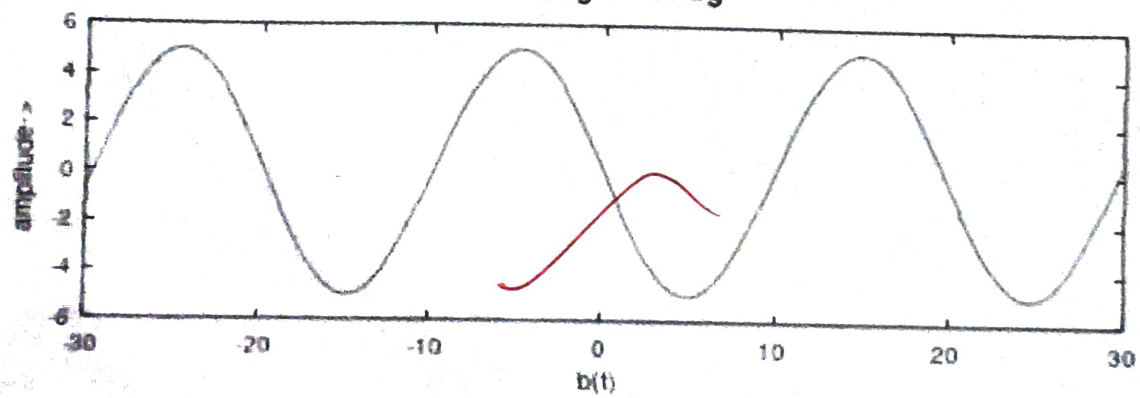
S.NO	DATE	EXPERIMENTS	PAGE NO	MARKS	SIGNATURE
01.	07/08/24	Generation of Elementary Discrete time sequence	01.	(10)	<i>[Signature]</i>
02	14/08/24	Linear and Circular Convolution	09	10	Day
03.	17/08/24	Cross Correlation and Auto Correlation	17	10	Day
04.	03/09/24	Frequency Analysis Using DFT	25	(10)	<i>[Signature]</i> 03/09/24
05.	24/09/24	Design of FIR filters and Demonstrates the filtering operation	31	(10)	<i>[Signature]</i> 24/09/24
06	15/10/24	Design of Butterworth & Chebyshev IIR Filters and Demonstrates the filtering operation	47	(10)	<i>[Signature]</i> 15/10/24

07	15.10.24	Linear Convolution From Circular Convolution	63	(9)	24 15/10/24
08	22.10.24	Interpolation and Decimation	75	(10)	24 22/10/24
09	22.10.24	overlap-Add Method Using Linear Convolution	85	(15)	24 22/10/24
10	5.11.24	Sampling rate conversion By a factor I/D	95	(10)	24 5/11/24

sine signal digital



sine signal analog



GENERATION OF ELEMENTARY DISCRETE TIME SEQUENCES

Aim:

To generate Discrete time sequences using MATLAB

Requirements:

- * Matlab Software
- * PC

Discrete Time:

The signals that are defined for discrete instant of time are known as discrete time signals. They are continuous in amplitude and discrete in time.

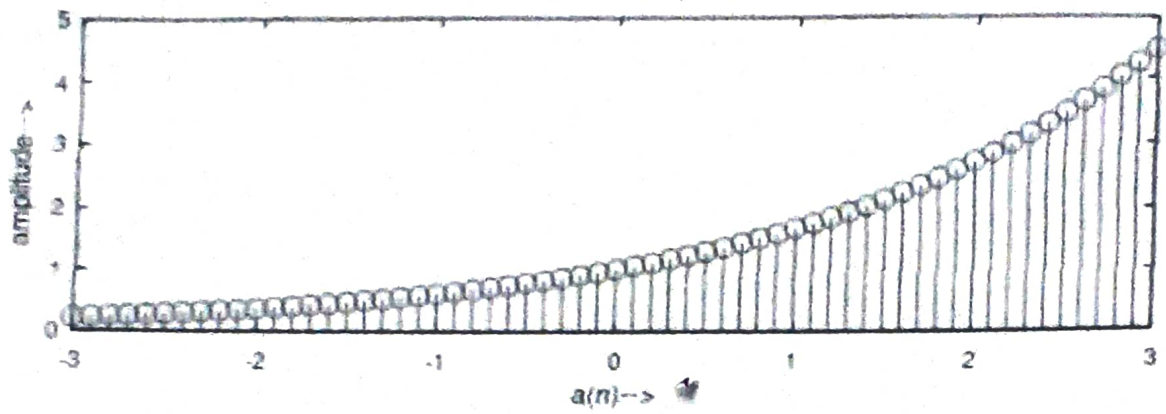
SINUSOIDAL SIGNAL:

It is mathematically expressed as

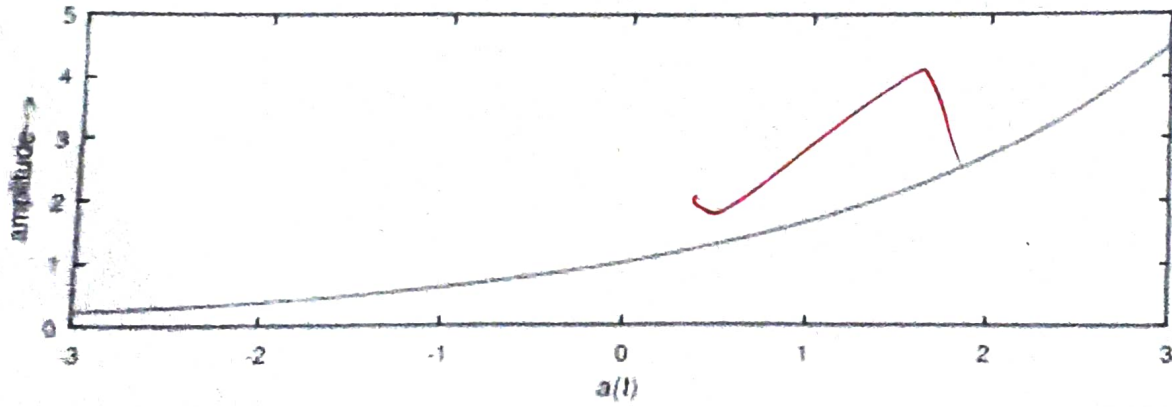
$$X(n) = \sin(2 * \pi * f * t)$$

Where f is the frequency, t is the Time.

exponential signal digital



exponential signal analog



EXPONENTIAL SIGNAL

The continuous time exponential signal can be expressed as follows

$$x(t) = C e^{at}$$

Where

C and a are in general numbers

DELTA FUNCTION

It is mathematically defined as the signal which has amplitude of one only at $t=0$.

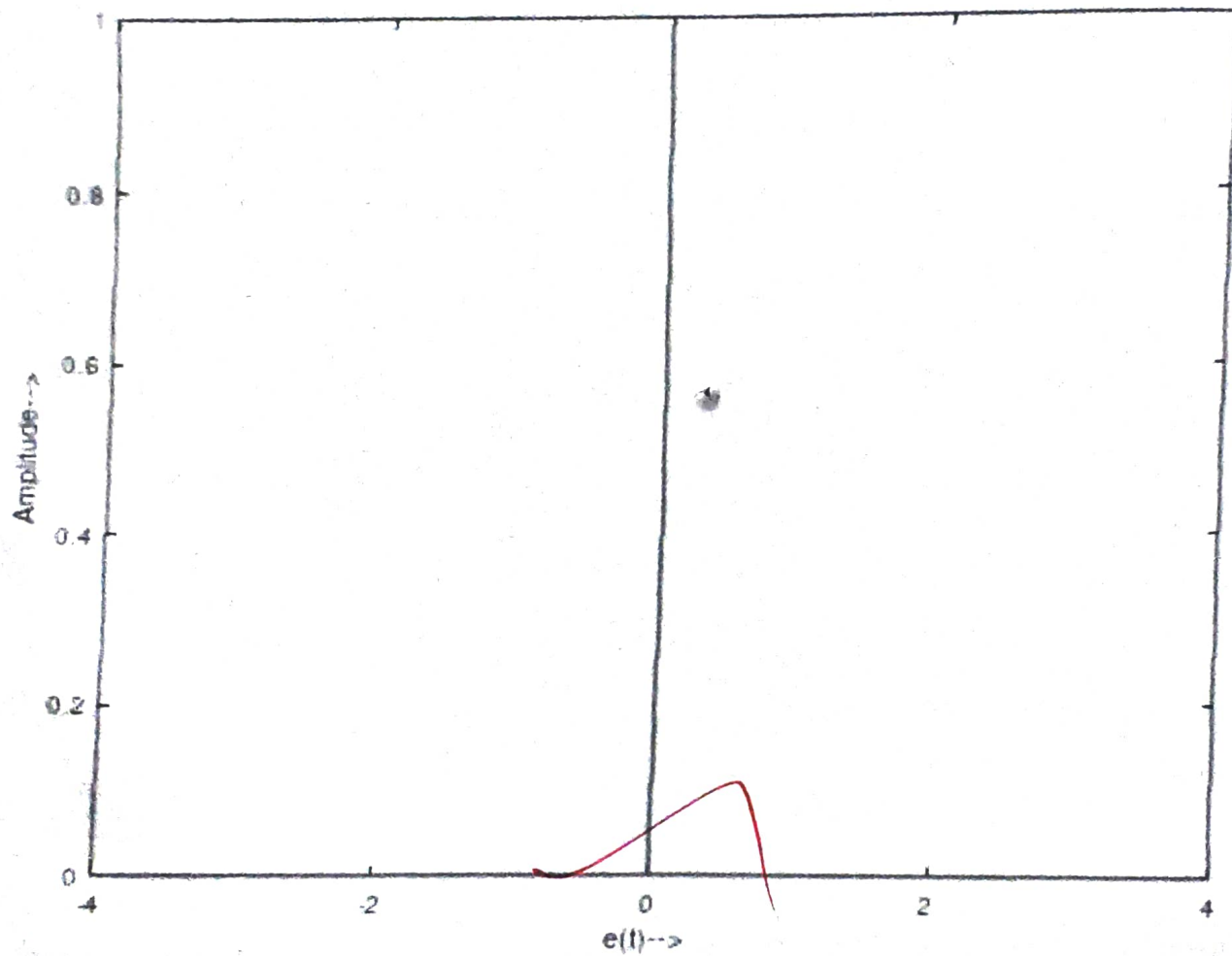
$$\delta(t) = 1 \text{ for } t=0$$

$$\delta(t) = 0 \text{ for } t \neq 0$$

Procedure:

- * Open the MATLAB software
- * Create the new empty file
- * Type the required program
- * Save it in the current directory
- * Save the output
- * End the program.

Delta signal



PROGRAM

Sine wave

```
X = -30:0.5:30;
```

```
Y = 5 * sin(2 * 3.14 * X * 100);
```

```
subplot(2,1,1); stem(X,Y)
```

```
xlabel('a(n) -->'); ylabel('amplitude -->')
```

```
title('sine signal digital');
```

```
subplot(2,1,2); plot(X,Y)
```

```
xlabel('b(t)'); ylabel('amplitude -->')
```

```
title('sine signal analog');
```

Exponential Signal:

```
X = -3:0.1:3;
```

```
y = exp(X/2);
```

```
subplot(2,1,1); stem(X,y)
```

```
xlabel('a(n) -->'); ylabel('amplitude -->')
```

```
title('exponential signal digital');
```

```
subplot(2,1,2); plot(X,y)
```

```
xlabel('a(t)'); ylabel('amplitude -->')
```

```
title('exponential signal analog');
```


Delta signal

$X = -4 : 0.01 : 4$

$Y = [x == 0]$

`plot(x,y); xlabel('t-->'); ylabel('Amplitude-->')`
`title('delta signal')`

~~Summary~~
7/8/24

Result:

Thus the MATLAB Code for generating sine, exponential and delta waveform (analog and digital) was performed and simulated and the output is obtained.

Output for Linear Convolution:

Enter the Sequence 1: [1 2 3 4 5 6 7]

Enter the Sequence 2: [1 2 3 4 5]

The resultant is:

1 4 10 20 35 50 65 72 70 58 35

Output for Circular Convolution:

Enter the Sequence 1: [1 2 3 4 5 6 7]

Enter the Sequence 2: [1 2 3 4 5]

The resultant is:

1 4 10 20 35 50 65

LINEAR AND CIRCULAR CONVOLUTION

Aim:

To write and simulate the MATLAB code for generating linear convolution and circular convolution.

Apparatus Required:

- * PC (Personal Computer)
- * MATLAB Software

Theory

CONVOLUTION

Convolution is a mathematical way of combining two signals to form a third signal.

LINEAR CONVOLUTION:

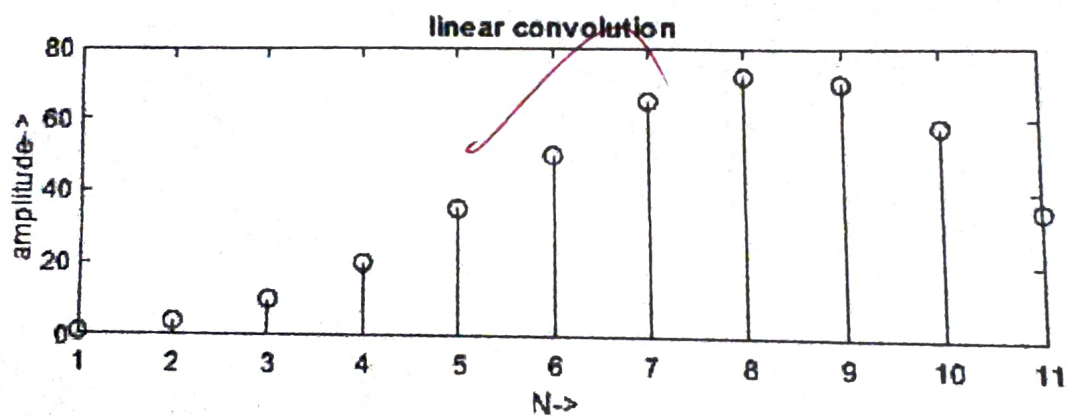
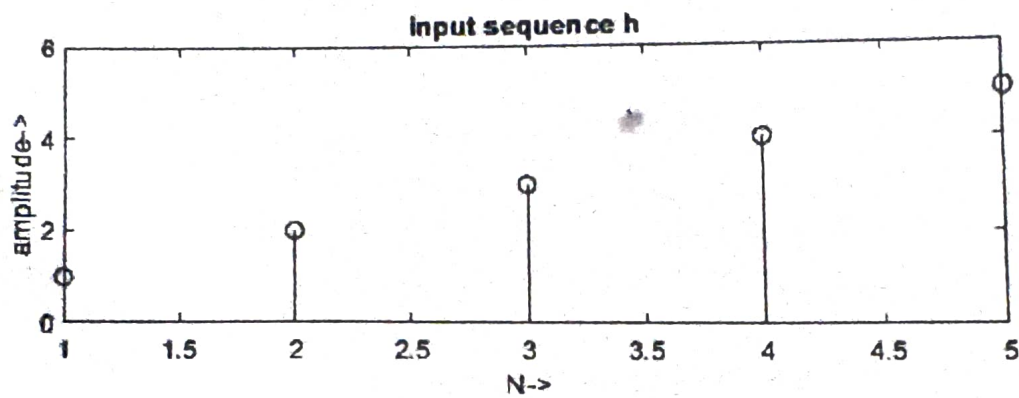
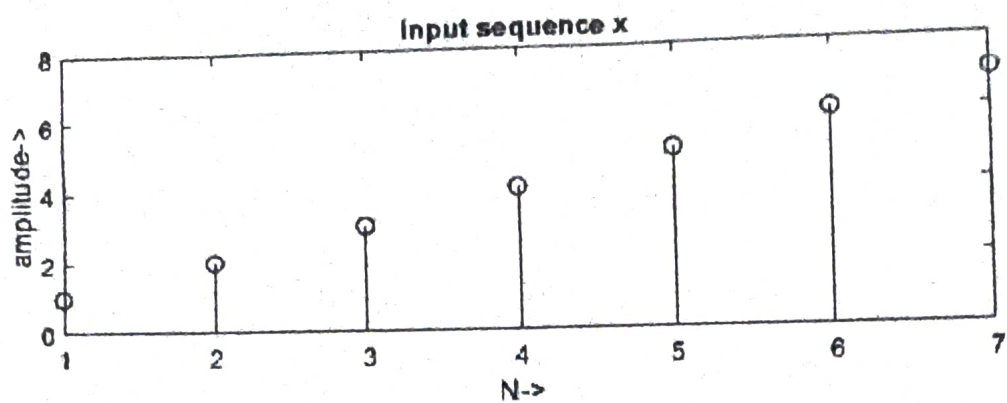
Linear Convolution is a mathematical operation used to express the relation between input and output of an LTI system.

$$Y(n) = X(n) * h(n)$$

Where $Y(n)$ = Output of LTI and

$X(n)$ = Input of LTI

Linear Convolution:



CIRCULAR CONVOLUTION:

The circular convolution is also known as "cyclic convolution" of two aperiodic function occurs when one of them is convolved in the normal way with a periodic summation of the other function. The DTFT of the product of two discrete sequences is the periodic convolution of DTFT of the individual sequences.

PROCEDURE:

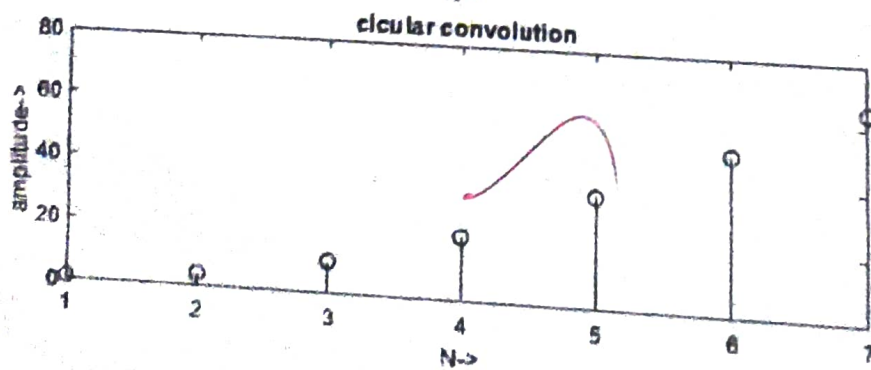
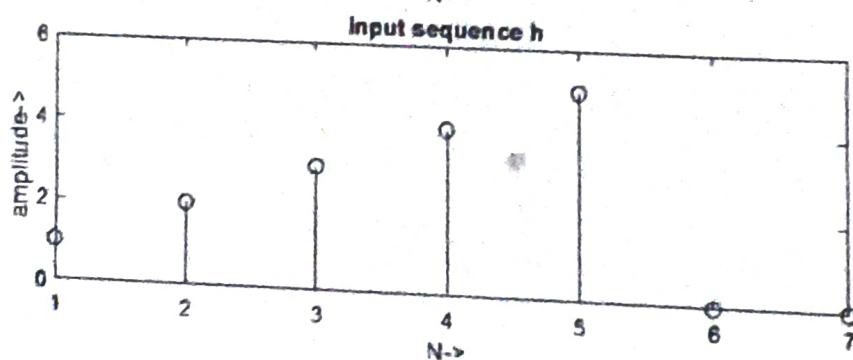
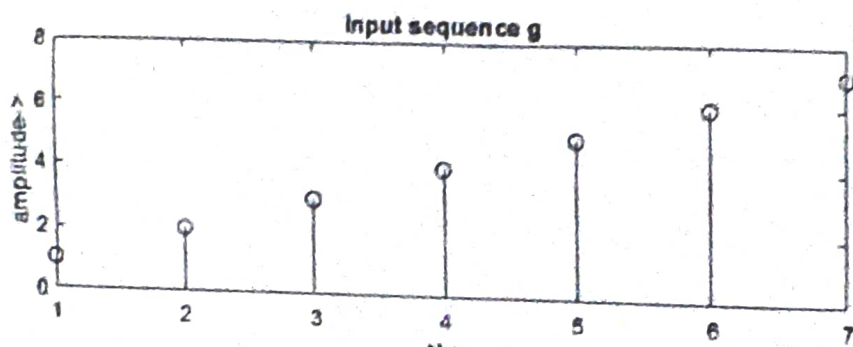
- * Open the MATLAB Software
- * Create the new empty file
- * Type the required program
- * Save it in the current directory
- * Save the output
- * End the program

PROGRAM

LINEAR CONVOLUTION

```
x=input('Enter the sequence 1:');  
h=input('Enter the sequence 2:');  
y=conv(x,h);  
subplot(3,1,1);  
stem(x);
```

Circular Convolution:




```

ylabel('amplitude -->');
xlabel('N-->');
title('Input Sequence x');
subplot(3,1,2);
stem(h);
ylabel('amplitude -->'); xlabel('N-->')
title('Input Sequence h');
subplot(3,1,3);
stem(y);
ylabel('amplitude -->'); xlabel('N-->')
title('linear convolution');

```

CIRCULAR CONVOLUTION

```

g=input('Enter the sequence 1:');
h=input('Enter the sequence 2:');
n1=length(g);
n2=length(h);
n=max(n1,n2);
n3=n1+n2;
if(n3>0)
    h=[h, zeros(1, n3)];
else
    g=[g, zeros(1, -n3)];
end
for n=1:n;
    y(n)=0;

```

```
for (p=1:n)
    i = n-i+1;
    if (pL=0
        i = n+1;
    end
    y(n) = [y(n) + (g(i) * h(p))];
end
end
disp('The resultant is'); y
stem(y);
xlabel('N-->');
ylabel('amplitud-->');
```

15/08/24

Result:

Thus the MATLAB code for linear convolution and circular convolution was performed and simulated and the output is obtained

CROSS CORRELATION AND AUTO CORRELATION

AIM:

To Write and stimulate the MATLAB Code for generating auto Correlation and Cross Correlation

APPARATUS REQUIRED:

- * PC (personal Computer)
- * MATLAB Software.

Theory:

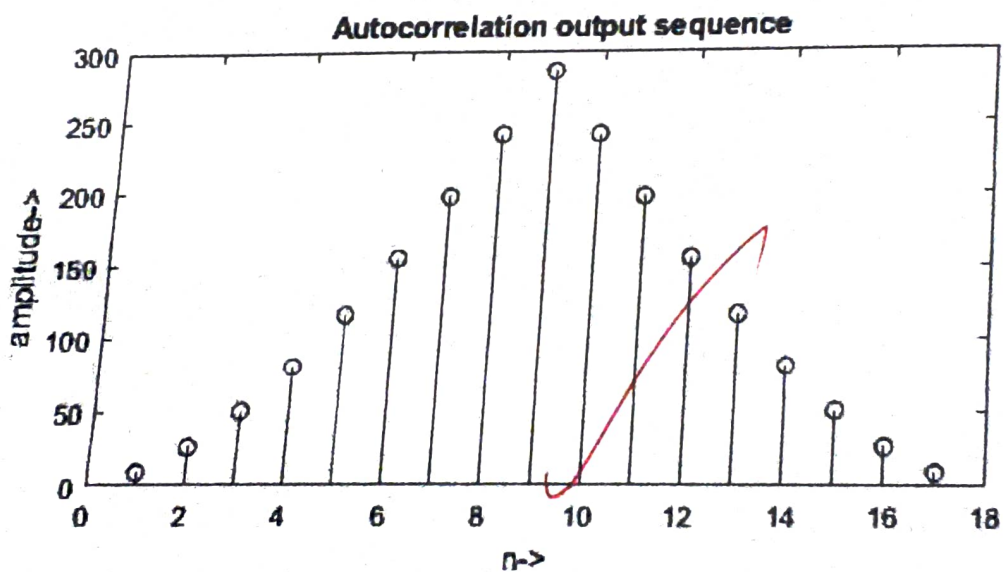
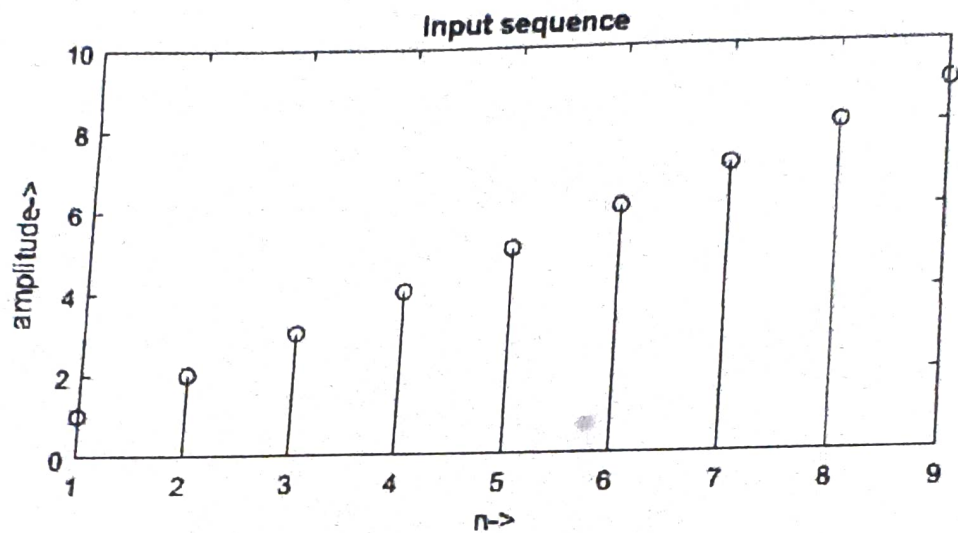
CORRELATION:

Correlation is a statistical measure that expresses the extent to which two variable are linearly related (meaning they change together at a constant rate.) It's a common tool for describing simple relationship without making a statement about cause and effect

DIFFERENCE BETWEEN AUTO CORRELATION AND CROSS CORRELATION:

Correlation is a statistical measure that expresses that extent to which two variables

Output for Autocorrelation:



Enter the input sequence 1: [1 2 3 4 5 6 7 8 9]
The resultant 16

9	26	50	80	115	154	196
240	285	240	196	154	115	80
50	26	9				

are linearly related (meaning they change together at a constant rate.) It's a common tool for describing simple relationship without making a statement about cause and effect.

PROCEDURE :

- * Open the MATLAB Software
- * Type the required program
- * Save it in the current directory
- * Save the output
- * End the program

PROGRAM

AUTO CORRELATION:

clc;

close all;

clear all;

x = input('Enter the sequence ('):');

y = xcorr(x, x);

Figure 1

subplot(2,1,1);

stem(x);

ylabel('Amplitude ->');

xlabel('n ->');

Title('Input sequence');

subplot(2,1,2);

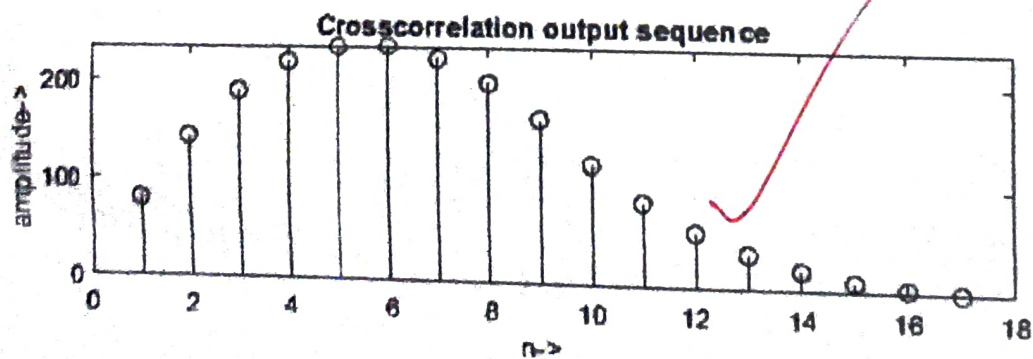
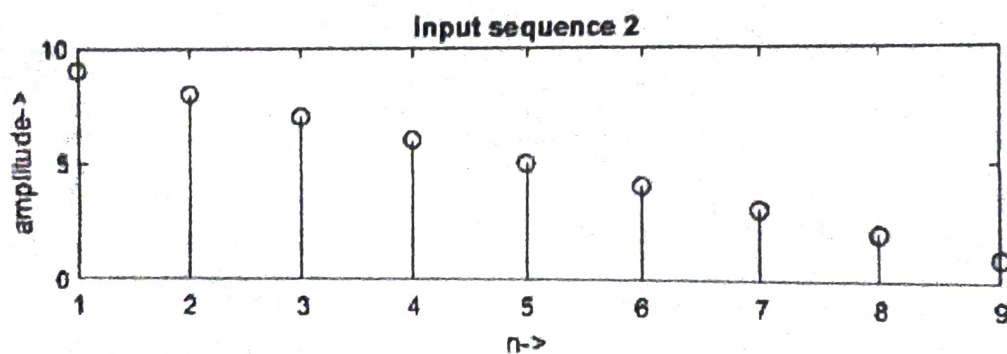
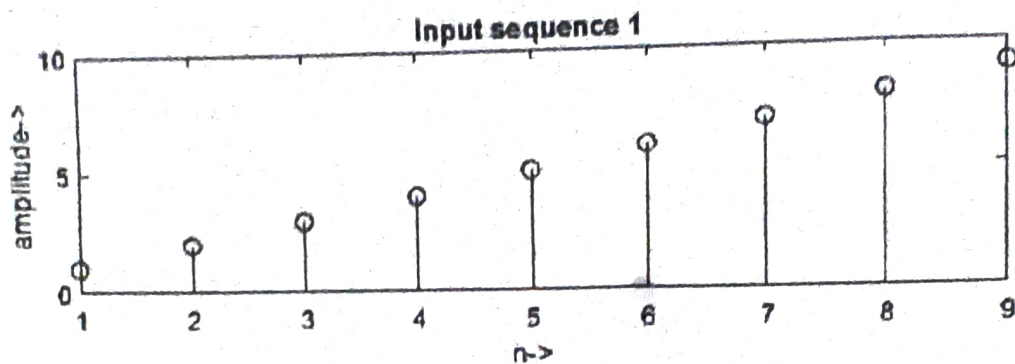
Output for Cross Correlation.

Enter the Sequence 1: [1 2 3 4 5 6 7 8 9]

Enter the Sequence 2: [9 8 7 6 5 4 3 2 1]

The resultant is

	16	1	4	10	20	35	56
		$n=17$	$n=16$	$n=15$			
	84	120	165	200	224	236	
	285	220	190	144	81		
		$n=3$	$n=2$	$n=1$			




```
stem(fliplr(y));  
ylabel('amplitude');  
xlabel('n-z');  
Title('output sequence');  
Disp('the resultant is');  
fliplr(y);
```

CROSS CORRELATION:

```
clc;  
clear all;  
close all;  
x = input('Enter the Sequence 1:');  
h = input('Enter the Sequence 2:');  
Y = Xcorr(x,h);  
Figure;  
subplot(3,1,1);  
stem(x);  
xlabel('n-z');  
ylabel('Amplitude ->');  
Title('Input Sequence 1');  
subplot(3,1,2);  
stem(fliplr(y));  
stem(h);  
xlabel('n-z');  
ylabel('Amplitude ->');  
Title('Input Sequence 2');
```

Disp('The resultant is');

Eliptr(y);

RESULT:

Thus the MATLAB code for generating correlation signal (analogy and digital) was performed and simulated and the output is obtained

FREQUENCY ANALYSIS USING DFT

AIM:

To write and simulate the MATLAB code for spectrum analysis using DFT.

APPARATUS REQUIRED:

- * PC (Personal Computer)
- * MATLAB Software.

Theory

DFT:

The discrete Fourier transform (DFT), implemented by one of the computationally efficient fast Fourier transform (FFT) algorithms, has become the core of many digital signal processing systems. These systems can perform general time domain signal processing and classical frequency domain processing.

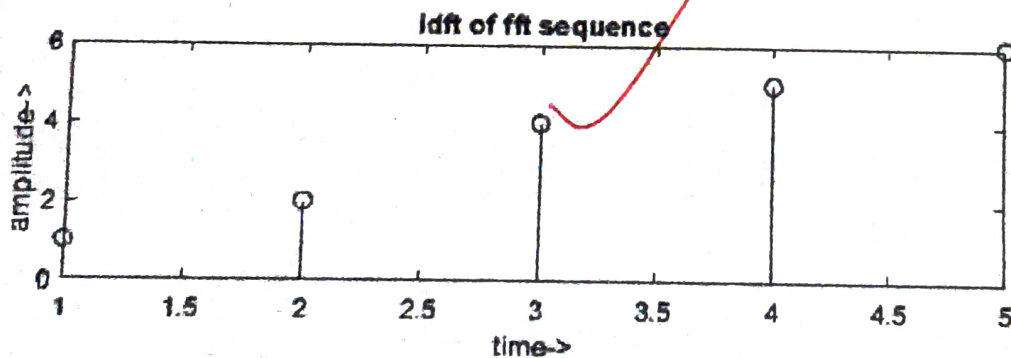
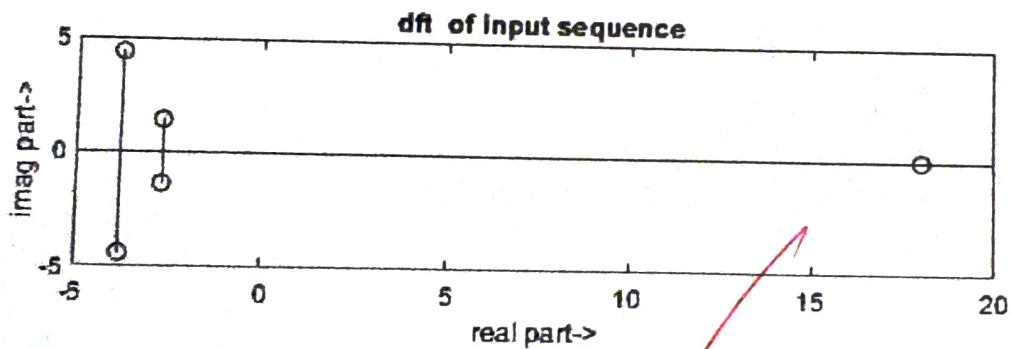
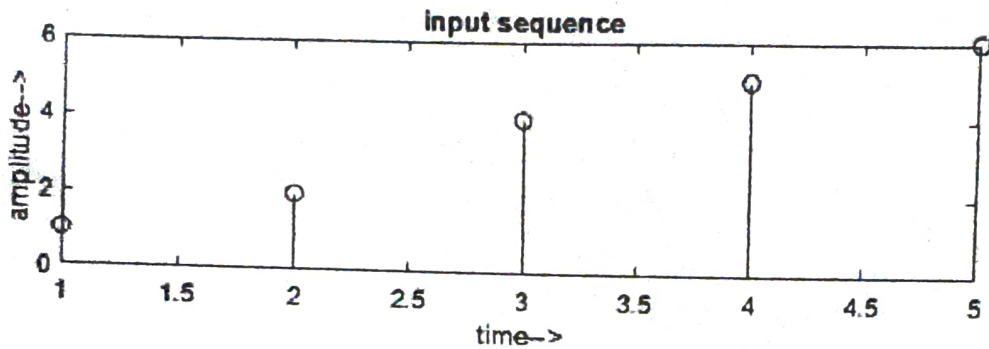
The goal of spectrum analysis is often to determine the frequency content of an analog (continuous time) signal. Very often, as in most modern spectrum analyzers, this is actually accomplished by sampling the analog signal.

Output:

Input Sequence : $x(t) = [1 \ 2 \ 4 \ 5 \ 6]$

Dft of Input Sequence : $y(k) = [18 + 10j, -3.809 + 4.392j,$
 $-2.691 + 1.4001j, -2.691 - 1.4001j,$
 $-3.809 - 4.392j]$

Idft of $y(k) = z(t) = [1 \ 2 \ 4 \ 5 \ 6]$



Windowing (truncating) the data, and Computing and Plotting the magnitude of its DFT.

PROCEDURE:

- * Open the MATLAB software
- * Create the new empty file
- * Type the required program
- * Save it in the current directory
- * save the output
- * End the program.

PROGRAM:

```
X = [1 2 4 5 6];  
subplot(3,1,1);  
stem(x); xlabel('Time ->') ylabel('amplitude ->')  
title('Input Sequence');  
Y = fft(X);  
subplot(3,1,2); disp(Y); xlabel('Real part ->'); ylabel  
('Imag part ->');  
stem(real(Y); imag(Y)); title('dft of Input Sequence');  
Z = ifft(Y);  
subplot(3,1,3); disp(Z); xlabel('real part ->');  
ylabel('imag part ->');  
stem(Z); title('idft of fft sequence')
```

~~Day~~
RESULT:

Thus the MATLAB code for spectrum analysis using DFT was performed and simulated and the output is obtained.

DESIGN OF FIR FILTERS (LPF/HPF/BPF/BSF) AND DEMONSTRATES THE FILTERING OPERATION

AIM:

To write and simulate the MATLAB code for FIR filter design.

APPARATUS REQUIRED:

FIR

Finite impulse response converts infinite number of sequences into a finite number of sequence. FIR filter design uses the following windows.

RECTANGULAR WINDOW:

The width of main lobe in window spectrum is $1/N$

In FIR filter designed using rectangular window the minimum stop band attenuation is 22db

$$w(n) = 1 \text{ for } -(N-1)/2 \leq n \leq (N-1)/2$$

$$w(n) = 0 \text{ for other } n$$

HANNING WINDOW:

The width of the main pole in window spectrum is $8/N$.

In FIR filter designed using rectangular window, the minimum stop band attenuation is 44 db.

$$W(n) = 0.5 - 0.5 \cos\left(\frac{2n}{N} - 1\right) \text{ for } -(N - \frac{1}{2}) \text{ to } (N - \frac{1}{2})$$

$$W(n) = 0 \text{ for other } n.$$

HAMMING WINDOW:

The width of the main pole in window spectrum is $8/N$.

In FIR filter designed using rectangular window, the minimum stop band attenuation is 51 db.

$$W(n) = 0.54 - 0.46 \cos\left(\frac{2n}{N} - 1\right) \text{ for } -(N - \frac{1}{2}) \text{ to } (N - \frac{1}{2})$$

$$W(n) = 0 \text{ for other } n.$$

LOW PASS FILTER:

The low pass filter allows the frequencies below the cutoff frequency and rejects the frequencies greater than the cutoff frequency.

HIGH PASS FILTER:

The high pass filter allows the frequencies below the cutoff frequency and rejects the frequencies lower than the cut off frequency.

BAND PASS FILTER:

The band pass filter passes the frequencies within the range ω_1 and ω_2 and rejects the frequencies lower outside the range.

BAND STOP FILTER or BAND REJECTION FILTER

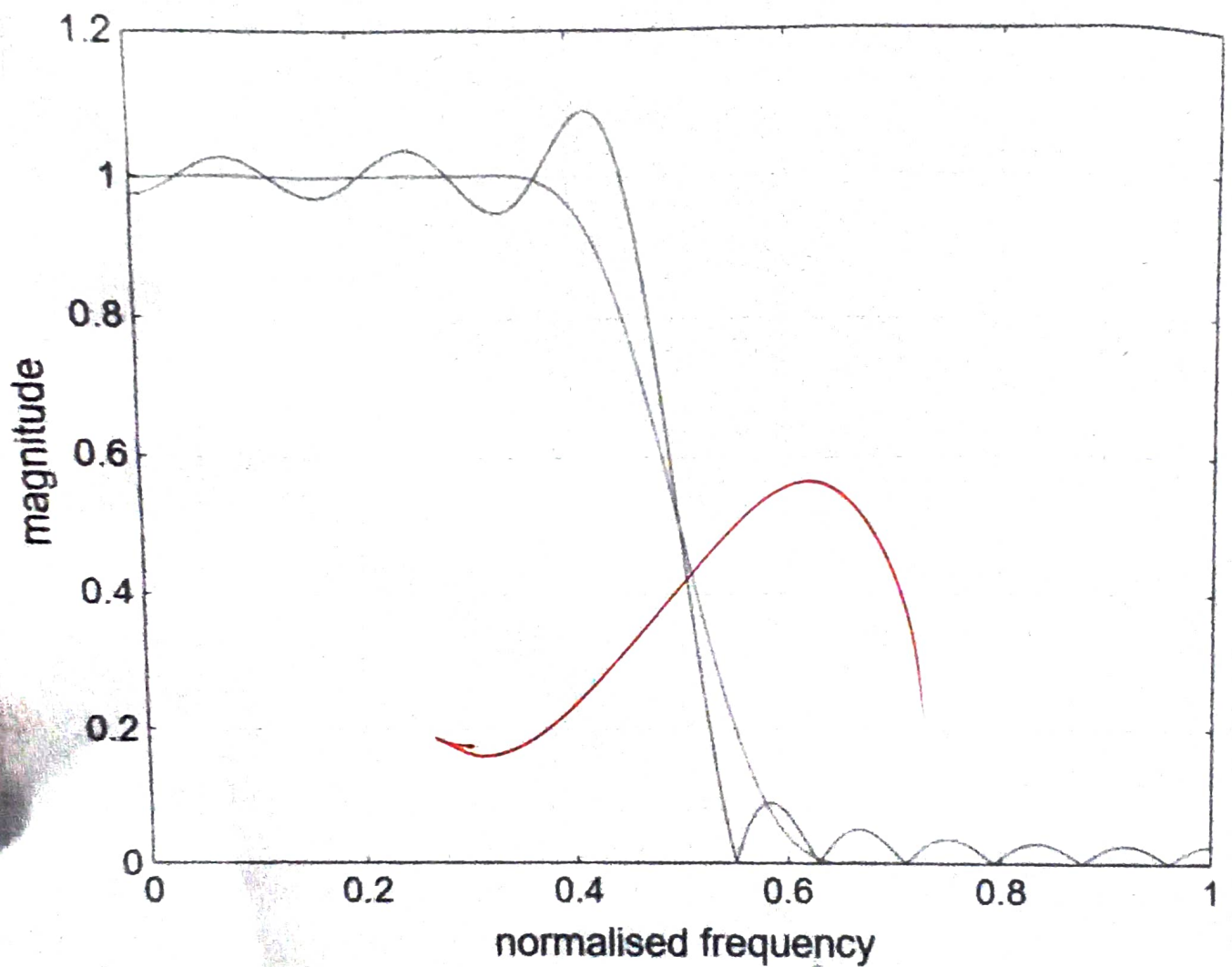
The band stop filter or the band rejection filter is a filter that passes most frequencies unaltered, but attenuates those in a specific range to very low levels. It is the opposite of a band pass filter. A notch filter is a band filter with narrow stop band.

PROCEDURE:

- * Open the MATLAB Software
- * Create the new empty file
- * Type the required program
- * Save it in the current directory
- * Save the output * End the program

Output :

Low pass filter:



PROGRAM

LOW PASS FILTER:

clc;

$W_c = 0.5 * \pi$;

$N = 25$;

$\alpha = (N-1)/2$;

$\epsilon = 0.001$;

$n = 0:1:N-1$;

$h_d = \sin(W_c * (n - \alpha + \epsilon)) ./ (\pi * (n - \alpha + \epsilon))$;

$W_x = \text{boxcar}(N)$;

$h_n = h_d .* W_x$;

$W = 0:0.01:\pi$;

$h = \text{freqz}(h_n, 1, W)$;

~~Plot~~ W / π , $\text{abs}(h)$;

~~xlabel('normalised frequency');~~

~~ylabel('Magnitude');~~

~~hold on;~~

$W_h = \text{hamming}(N)$;

$h_n = h_d .* W_h$;

$W = 0:0.01:\pi$;

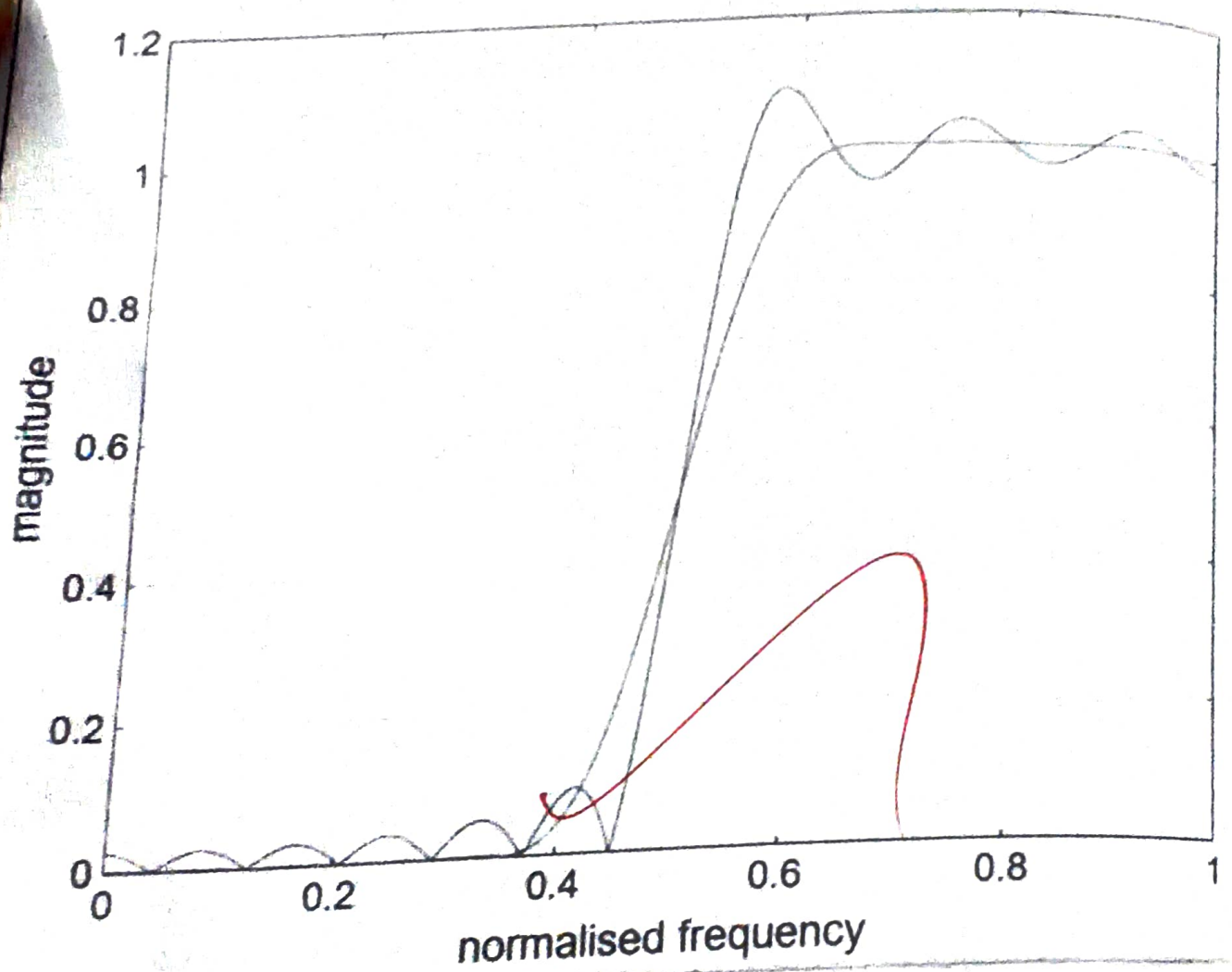
$h = \text{freqz}(h_n, 1, W)$;

$\text{Plot}(W / \pi, \text{abs}(h), '-')$;

grid;

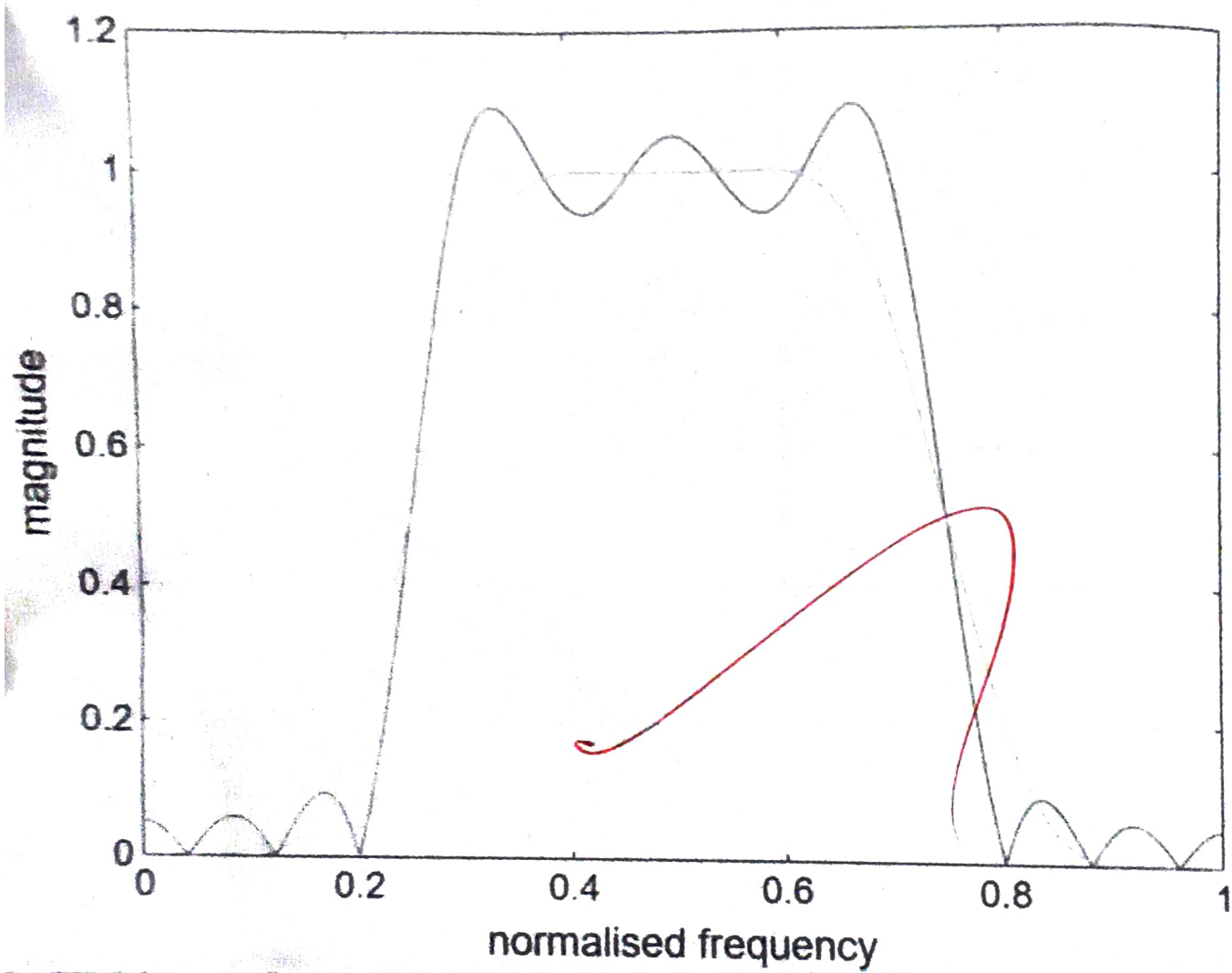
hold off;

High pass filter



HIGH PASS FILTER: $C/c;$ $W_c = 0.5 * P_i;$ $N = 25;$ $\alpha = (N-1)/2;$ $\epsilon = 0.001;$ $n = 0:1:N-1;$ $h_d = (\sin(P_i * (n - \alpha + \epsilon)) - \sin(W_c * (n - \alpha + \epsilon))) /$
 $(P_i * (n - \alpha + \epsilon));$ $W_r = \text{boxcar}(N);$ $h_n = h_d * W_r';$ $W = 0:0.01:P_i;$ $h = \text{freqz}(h_n, 1, W);$ $\text{Plot}(W/P_i, \text{abs}(h));$ $\text{xlabel}('normalised frequency');$ $\text{ylabel}('magnitude');$ $\text{hold on};$ $W_h = \text{hanning}(N);$ $h_n = h_d * W_h';$ $W = 0:0.01:P_i$ $h = \text{freqz}(h_n, 1, W);$ $\text{Plot}(W/P_i, \text{abs}(h), '-');$ $\text{grid};$ $\text{hold off};$

Band pass Filter:



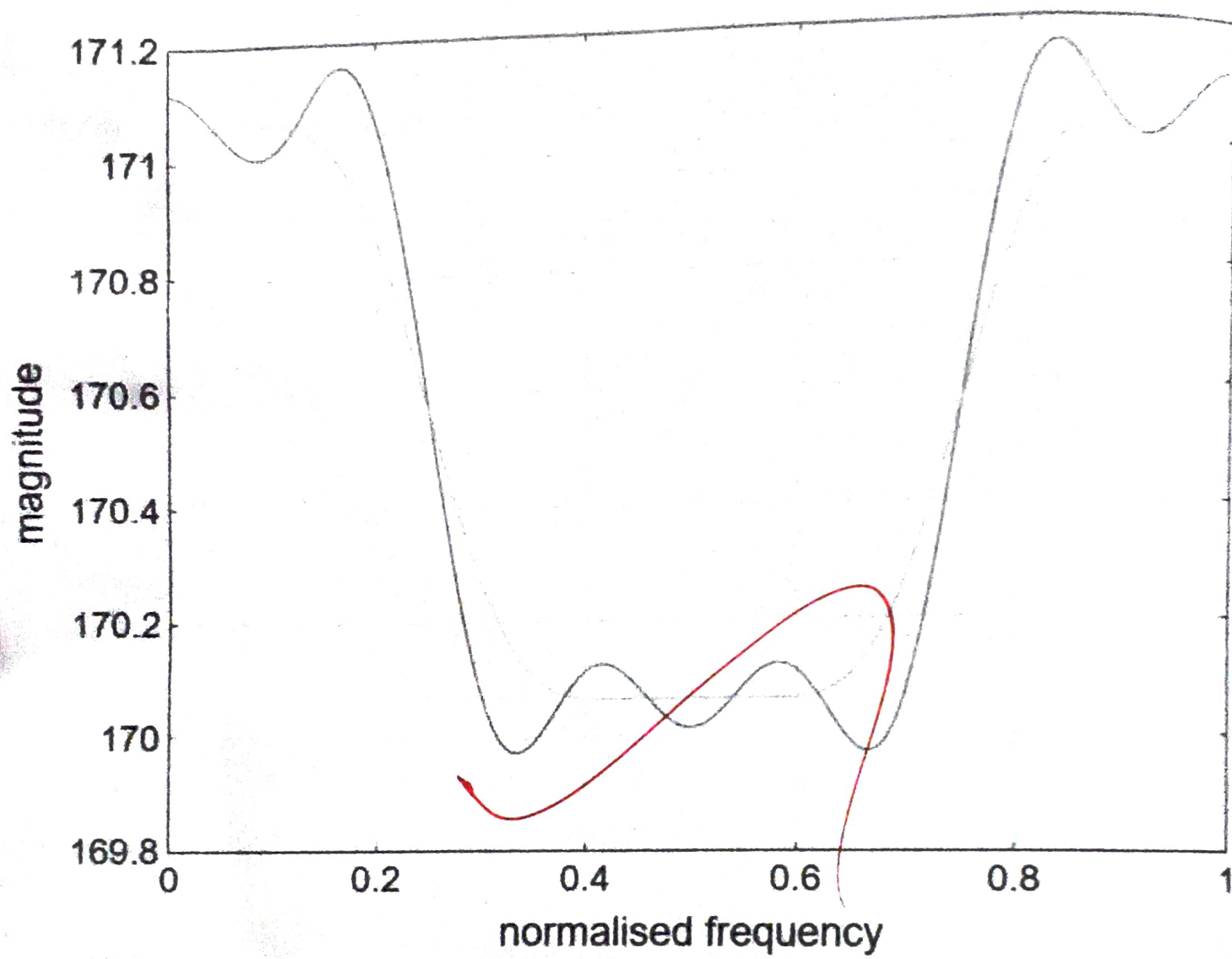
BAND PASS FILTER:

clc;

 $Wc1 = 0.25 * \pi;$ $Wc2 = 0.75 * \pi;$ $N = 25; \alpha = (N-1)/2;$ $\epsilon = 0.001;$ $n = 0:1:N-1;$

$$hd = (\sin(Wc2 * (n - \alpha + \epsilon)) - \sin(Wc1 * (n - \alpha + \epsilon))) / (\pi * (n - \alpha + \epsilon));$$
 $Wl = \text{boxcar}(N);$ $hn = hd * Wl';$ $Wl = 0:0.01:\pi;$ $h = \text{freqz}(hn, 1, Wl);$ $\text{Plot}(W/\pi, \text{abs}(h));$ $\text{xlabel}(' \text{normalised frequency} ');$ $\text{ylabel}(' \text{magnitude} ');$ $\text{hold on};$ $Wl_h = \text{hamming}(N);$ $hn = hd * Wl_h';$ $Wl = 0:0.01:\pi;$ $h = \text{freqz}(hn, 1, Wl);$ $\text{Plot}(W/\pi, \text{abs}(h), '-');$ $Wl_h = \text{hamming}(N);$ $hn = hd * Wl_h';$ $Wl = 0:0.01:\pi;$

Band Stop Filter:




```
h = freqz(hn, 1, W);  
Plot (W / Pi, abs(h), '—');  
grid;  
hold off;
```

BAND STOP FILTER:

```
clc;  
Wc1 = 0.25 * Pi;  
Wc2 = 0.75 * Pi;  
N = 25;  
alpha = (N-1)/2;  
Eps = 0.001;  
n = 0:1:N-1;  
hd = (sin(Wc1 * (n-alpha+eps)) - sin(Wc2 * (n-alpha+eps))  
      + sin(Pi * n - alpha + eps)) / (Pi * (n-alpha+eps));  
Wx = boxcar(N);  
hn = hd * Wx';  
Wl = 0.01 0.01 * Pi;  
h = freqz(hn, 1, W);  
Plot (W / Pi, abs(h));  
xlabel('normalised frequency');  
ylabel('Magnitude');  
hold on;  
Wh = hamming(N);  
hn = hd * Wh';  
Wl = 0:0.01:Pi;
```

```
h = freqz(hn, 1, W);  
Plot (W/pi, abs(h), '-');  
Wb = hamming(N);  
hn = hd.*Wb;  
W = 0:0.01:pi;  
h = freqz(hn, 1, W);  
Plot (W/pi, abs(h), '-');  
grid;  
hold off;
```

22/10/24

RESULT:

Thus the MATLAB for FIR filter design was performed and stimulated and the output is obtained.